República Bolivariana de Venezuela

Ministerio del Poder Popular para la Educación Universitaria

Universidad Politécnica Territorial

´´José Antonio Anzoátegui´´

Trayecto 1 Fase 2 PNF: Informática

Municipio Guanta, Edo. Anzoátegui

“INFORME REPASO”

Profesor:

Lenis Humner

Estudiante:

Rubén Díaz

C.I: 32.500.566

Guanta, 8 de noviembre de 2026

1) FUNDAMENTOS POO (PROGRAMACION ORIENTADAS A OBJETOS)

Es un paradigma de programación que utiliza "objetos" y sus interacciones para diseñar aplicaciones y programas informáticos. Está construida sobre cuatro pilares fundamentales que permiten organizar el código de manera modular, reutilizable y mantenible.

- **Clase:** Es una plantilla o molde que define los atributos (variables o propiedades) y los métodos (funciones o comportamientos) que tendrán los objetos creados a partir de ella.

- **Objeto:** Es una instancia concreta de una clase. Representa una entidad del mundo real o del dominio del problema con un estado (valores de sus atributos) y un comportamiento específico.

Abstracción

Consiste en aislar los elementos esenciales de un objeto, ignorando los detalles irrelevantes o complejos. Permite modelar entidades del mundo real centrándose en lo que hace el objeto y qué información maneja, exponiendo solo una interfaz simplificada.

Encapsulamiento

Es el mecanismo que agrupa los datos (atributos) y los métodos que operan sobre esos datos dentro de una misma estructura (la clase), restringiendo el acceso directo desde fuera mediante modificadores de visibilidad:

- **`public`:** Accesible desde cualquier lugar.

- **`private`:** Accesible únicamente dentro de la propia clase.

- **`protected`:** Accesible dentro de la clase y sus clases derivadas (herencia).

Herencia

Permite que una clase nueva (subclase o clase hija) adopte los atributos y métodos de una clase existente (superclase o clase padre). Esto fomenta la reutilización de código y establece jerarquías lógicas (por ejemplo, una clase `Vehículo` de la cual heredan `Auto` y `Moto`).

Polimorfismo

Literalmente significa "muchas formas". Permite que objetos de diferentes clases respondan al mismo mensaje o método de formas distintas. Se implementa principalmente mediante la sobreescritura de métodos (*method overriding*), donde una clase hija redefine un comportamiento heredado de su padre.

2) UML (Lenguaje Unificado de Modelado) (Diagrama)

Es el estándar de la industria para visualizar, especificar, construir y documentar los artefactos de un sistema de software orientado a objetos. Permite plasmar la arquitectura y el diseño antes de escribir código, facilitando la comunicación entre desarrolladores y analistas.

Tipos de Diagramas en UML

1. Diagramas Estructurales (Estáticos)

Muestran la estática del sistema, es decir, las partes físicas o conceptuales que permanecen sin importar el tiempo.

- **Diagrama de Clases (El más importante en POO):**

  Muestra las clases del sistema, sus atributos, operaciones (métodos) y las relaciones estáticas entre ellas (asociación, herencia, agregación, composición).

- **Diagrama de Objetos:**

  Muestra instancias concretas de las clases (objetos) y sus valores en un momento específico del tiempo. Útil para verificar la estructura de clases compleja.

- **Diagrama de Componentes:**

  Muestra la organización y las dependencias entre los componentes de software (módulos, librerías, ejecutables).

- **Diagrama de Despliegue:**

  Muestra la arquitectura física del hardware (servidores, dispositivos, nodos) y dónde se ejecutan los componentes de software.

2. Diagramas de Comportamiento (Dinámicos)

Muestran la dinámica del sistema, cómo cambia el estado o cómo interactúan los objetos a lo largo del tiempo.

- **Diagrama de Casos de Uso:**

Modela la funcionalidad del sistema desde el punto de vista del usuario o actor externo. Muestra quién interactúa con qué (*Actores* y *Casos de Uso*).

- **Diagrama de Secuencia:**

Muestra la interacción entre objetos a lo largo del tiempo, detallando el orden cronológico de los mensajes (llamadas a métodos) que se intercambian.

- **Diagrama de Actividades:**

Esencialmente un diagrama de flujo avanzado que muestra flujos de trabajo paso a paso, decisiones, bifurcaciones y concurrencia (procesos de negocio o algoritmos complejos).

- **Diagrama de Estados (Máquina de Estados):**

Muestra los diferentes estados por los que pasa un objeto a lo largo de su ciclo de vida y los eventos que provocan las transiciones entre un estado y otro.

3) Clases y Objetos

Una clase es una plantilla abstracta, un plano arquitectónico o un diseño conceptual. No ocupa espacio físico en la memoria de la computadora para almacenar datos reales; simplemente define cómo serán los elementos que se crearán a partir de ella.

Una clase agrupa dos cosas principales:

- **Atributos (Variables):** Representan el estado o las características (ej. marca, color, velocidad actual).

- **Métodos (Funciones):** Representan el comportamiento o las acciones que pueden realizar (ej. acelerar, frenar, tocar Bocina).

Un objeto es una instancia concreta de una clase. Es el momento en el que el plano de papel cobra vida y se convierte en una entidad real en la memoria RAM con la que el programa puede interactuar.

Siguiendo la analogía:

- La clase es el plano arquitectónico de un edificio.

- El objeto es el edificio construido en la calle (puedes construir múltiples edificios idénticos usando el mismo plano, pero cada uno tiene su propia ubicación y residentes).

4) Especificadores

Son palabras clave que definen el ámbito y las reglas de visibilidad de los miembros de una clase (es decir, sus atributos y métodos).

Funcionan como las "puertas y cerraduras" que controlan qué partes del código pueden interactuar con los datos internos de un objeto y qué partes tienen prohibido el acceso, garantizando así el principio de Encapsulamiento.

1. `public` (Público)

- **Alcance:** Total y sin restricciones.

- **Comportamiento:** Un atributo o método declarado como `public` puede ser accedido, leído y modificado desde cualquier lugar: dentro de la misma clase, desde clases hijas (herencia), desde otras clases no relacionadas y directamente desde fuera del objeto (instancia).

- **Cuándo usarlo:** Para métodos que forman la interfaz pública de la clase (las acciones que los usuarios o sistemas externos deben poder ejecutar libremente).

2. `private` (Privado)

- **Alcance:** Estrictamente limitado.

- **Comportamiento:** Un miembro `private` solo puede ser accedido desde dentro de la misma clase que lo declaró. Ninguna otra clase externa, ni siquiera sus propias clases hijas (herencia), pueden verlo o modificarlo directamente.

- **Cuándo usarlo:** Para proteger el estado interno (atributos sensibles) y detalles de implementación que no deben ser manipulados de forma imprudente por el exterior. Para exponerlos de forma segura, se usan métodos públicos de apoyo (Getters y Setters).

3. `protected` (Protegido)

- **Alcance:** Intermedio (Clase base + Jerarquía de herencia).

- **Comportamiento:** Es idéntico a `private` para el mundo exterior (no se puede acceder desde fuera de las clases), pero sí permite el acceso a las clases hijas (subclases) a través de la herencia.

- **Cuándo usarlo:** Cuando estás diseñando una jerarquía de clases y quieres que los elementos hijos (subclases) hereden y utilicen directamente ciertos atributos o métodos internos del padre, pero manteniendo ocultos esos datos para el resto de la aplicación.

5) Constructores

Es un método especial dentro de una clase que se ejecuta de forma automática e instantánea en el preciso momento en que se crea (instancia) un objeto mediante la palabra clave

El objetivo fundamental de un constructor es inicializar el estado del objeto. Garantiza que el objeto "nazca" con los valores y datos obligatorios necesarios para operar correctamente, evitando que las propiedades queden vacías o en un estado inconsistente.

Si no defines un constructor de forma explícita, los lenguajes suelen proveer un constructor por defecto (vacío), pero escribir el tuyo propio te da el control total sobre los requisitos de creación.

Características Clave de los Constructores

- **Ejecución Automática:** No necesitas invocarlo manualmente como a un método común (prohibido hacer algo como `$objeto->__construct()`); el sistema lo dispara por debajo de la cuerda al instanciar.

- **Sin Tipo de Retorno:** A diferencia de los métodos habituales, los constructores no devuelven ningún valor con la sentencia `return`.

- **Nombre Reservado o Coincidente:**

En **PHP** se declara obligatoriamente como `__construct()`.

En **Java, C++ y C#** se llama exactamente igual que la clase.

En **Python** se define como `__init__()`.

6) Herencia

Es uno de los cuatro pilares fundamentales de la Programación Orientada a Objetos (POO). Es un mecanismo que permite crear una clase nueva (*subclase* o *clase hija*) a partir de otra clase ya existente (*superclase* o *clase padre*).

La clase hija hereda automáticamente todos los atributos y métodos públicos y protegidos de la clase padre, lo que evita la duplicación de código y permite establecer una jerarquía lógica de tipo "es un" (por ejemplo, un `Gerente` *es un* `Empleado`).

- **Clase Padre (Superclase):** Es la clase general que contiene los atributos y comportamientos comunes.

- **Clase Hija (Subclase):** Es la clase especializada que hereda del padre y puede añadir sus propios atributos, sus propios métodos, o modificar los que ya heredó.

- **Reutilización y Extensibilidad:** En lugar de reescribir la misma lógica para diferentes tipos de objetos, defines la base una sola vez y extiendes las capacidades según sea necesario.

7) Clases Abstractas

Es un tipo especial de clase dentro de la Programación Orientada a Objetos que está diseñada exclusivamente para funcionar como una plantilla o base para otras clases.

Su característica principal es que no se puede instanciar directamente (es decir, no puedes hacer `new MiClaseAbstracta()`), y su propósito es unificar comportamientos comunes mientras obliga a las clases hijas a implementar detalles específicos.

Características Principales

- **No Instanciable:** Intentar crear un objeto directamente a partir de una clase abstracta genera un error en tiempo de ejecución o compilación.

- **Métodos Abstractos:** Una clase abstracta puede contener métodos abstractos. Estos son métodos declarados *sin cuerpo* (sin código interno), que funcionan como un contrato obligatorio: cualquier clase hija que herede de la clase abstracta está obligada a escribir la lógica de ese método.

- **Métodos Concretos:** También pueden contener métodos normales con lógica implementada, los cuales son heredados y utilizados directamente por las clases hijas sin necesidad de reescribirlos.

8) Polimorfismo

La palabra Polimorfismo viene del griego *poly* (muchos) y *morph* (formas). En el desarrollo de software, describe la capacidad de diferentes objetos o clases para responder a una misma llamada de método o mensaje, pero adaptando su comportamiento interno a su propia naturaleza.

Te permite programar pensando en interfaces generales en lugar de preocuparte por los detalles específicos de cada implementación particular.

Polimorfismo en Tiempo de Ejecución (Dinámico / Sobrescritura)

Es el que se logra mediante la herencia y las clases abstractas o interfaces. Una clase hija redefine (*method overriding*) un método heredado de su padre.

- El lenguaje determina en tiempo de ejecución qué versión del método debe dispararse según el tipo real del objeto que recibió la instrucción, y no según el tipo de la variable que lo apunta.

Polimorfismo en Tiempo de Compilación (Estático / Sobrecarga)

Ocurre cuando tienes varios métodos con exactamente el mismo nombre dentro de una misma clase, pero con diferentes firmas (es decir, distinta cantidad de parámetros o diferentes tipos de datos).

- Lenguajes como PHP no soportan la sobrecarga clásica de métodos de forma nativa, pero lenguajes fuertemente tipados como Java, C++ o C# la utilizan ampliamente.

9) Interfaces

Es uno de los conceptos más potentes de la Programación Orientada a Objetos (POO). Funciona como un contrato formal o una especificación estricta que define *qué* debe hacer una clase, pero sin especificar *cómo* debe hacerlo.

Mientras que una clase abstracta comparte código y lógica base, una interfaz solo define firmas de métodos (el nombre, los parámetros y el tipo de retorno), obligando a cualquier clase que la implemente a escribir la lógica interna por sí misma.

- **Contrato Obligatorio:** Si una clase decide implementar una interfaz, está firmando un contrato legal ante el compilador: debe implementar *todos y cada uno* de los métodos definidos en ella, o el programa fallará.

- **Métodos sin Cuerpo:** Por definición conceptual, los métodos de una interfaz no tienen bloques de código `{}` (salvo en características muy avanzadas de algunos lenguajes modernos). Solo declaran la firma.

- **Herencia Múltiple de Tipo:** A diferencia de las clases (donde solo puedes heredar de un único padre), una clase puede implementar múltiples interfaces, combinando diferentes comportamientos de forma limpia.

- **Visibilidad Pública Implícita:** Todos los métodos declarados en una interfaz son automáticamente `public`.

10) Arquitectura Web

Es el diseño estructural y la organización de los componentes de un sistema de software que opera a través de la web. Define cómo interactúan el cliente (el navegador o la interfaz de usuario), el servidor, las bases de datos y los servicios externos para procesar peticiones y entregar respuestas de manera eficiente, segura y escalable.

El Modelo Fundamental: Cliente-Servidor

- **El Cliente (Front-end):** Es la interfaz con la que interactúa el usuario final (ej. el navegador web ejecutando HTML, CSS y JavaScript). Es responsable de capturar las acciones del usuario y renderizar los datos visualmente.

- **El Servidor (Back-end):** Es el sistema centralizado o distribuido que procesa la lógica de negocio, se conecta a las bases de datos, valida la seguridad y genera la respuesta que se enviará de vuelta al cliente.

Arquitectura por Capas (N-Tier Architecture)

Para mantener el código organizado y fácil de mantener, las aplicaciones web se estructuran habitualmente en capas lógicas:

1. **Capa de Presentación (Interfaz):** Lo que ve el usuario (vistas, componentes visuales, formularios).

2. **Capa de Lógica de Negocio (Back-end / Application Layer):** Donde residen las reglas de la aplicación, el procesamiento de datos, validaciones y la toma de decisiones (aquí es donde la POO y los patrones de diseño brillan con mayor intensidad).

3. **Capa de Datos (Persistencia):** El motor de base de datos (como MySQL, PostgreSQL) y los sistemas de almacenamiento donde se guardan de forma permanente los registros.

El Patrón MVC (Model-View-Controller)

En el desarrollo web basado en frameworks modernos (como Laravel en PHP), el patrón MVC es el estándar arquitectónico por excelencia para estructurar la lógica del servidor:

- **Modelo (Model):** Representa la capa de datos y la lógica de negocio pura. Se encarga de comunicarse con la base de datos, aplicar reglas sobre los datos y estructurarlos.

- **Vista (View):** Es la interfaz visual que recibe los datos procesados y los muestra al usuario (plantillas HTML, JSON para APIs, etc.).

- **Controlador (Controller):** Actúa como el puente o intermediario. Recibe la petición HTTP del usuario, le pide al Modelo los datos necesarios y se los entrega a la Vista para que los renderice.

Estructuras de Aplicación: Monolito vs. Microservicios

A nivel de arquitectura de sistemas, existen dos enfoques principales para empaquetar y desplegar una aplicación web:

- **Arquitectura Monolítica:**

¿Qué es?: Toda la aplicación (interfaz, lógica de negocio, conexión a base de datos) se construye y se despliega como una única unidad o paquete de software.

Ventajas: Sencilla de desarrollar, probar y desplegar en las primeras fases (ideal para la mayoría de proyectos académicos, medianos o MVPs).

- **Arquitectura de Microservicios:**

¿Qué es?: El sistema se divide en pequeños servicios independientes y autónomos que se comunican entre sí a través de redes (usualmente mediante APIs REST o colas de mensajes). Cada microservicio cumple una única función de negocio.

Ventajas: Alta escalabilidad y tolerancia a fallos; si un servicio cae, el resto de la aplicación sigue funcionando. Sin embargo, introduce una complejidad operacional muy alta.

El Entorno de Ejecución (Pila Tecnológica Tradicional)

En la práctica, una arquitectura web estándar de tipo LAMP/LEMP (como la que se utiliza habitualmente en entornos de desarrollo local con XAMPP y servidores de producción) integra los siguientes componentes físicos y lógicos:

- **Servidor Web (Ej. Apache o Nginx):** Escucha las peticiones HTTP entrantes desde internet y dirige el tráfico hacia la aplicación.

- **Intérprete / Lenguaje (Ej. PHP):** Procesa la lógica dinámica del servidor.

- **Sistema Gestor de Bases de Datos (Ej. MySQL):** Almacena y administra la información de forma relacional.

11) Diseño (HTML/CSS)

Es la combinación de tecnologías que permite estructurar y dar estilo visual a una página o aplicación web para que los usuarios puedan interactuar con ella de forma clara, funcional y atractiva. Constituye la base del Front-end (el lado del cliente).

HTML: La Estructura y el Esqueleto

HTML no es un lenguaje de programación, sino un lenguaje de marcado de hipertexto. Su función es definir el contenido y la jerarquía semántica de una página web utilizando etiquetas (`tags`).

- **Semántica Web:** Utilizar las etiquetas correctas no solo es una buena práctica, sino que es vital para la accesibilidad (lectores de pantalla) y el posicionamiento en buscadores (SEO). En lugar de usar contenedores genéricos (`<div>`) para todo, se emplean elementos con significado real:

  `<header>`: Cabecera de la página o sección.

  `<nav>`: Barra de navegación principal.

  `<main>`: El contenido principal y único de la página.

  `<section>` y `<article>`: Agrupaciones temáticas de contenido.

- **El DOM (Document Object Model):** Cuando el navegador lee el código HTML, construye un árbol de nodos en la memoria llamado DOM, que es la estructura interactiva que JavaScript y CSS pueden manipular en tiempo real.

2. CSS: El Estilo, el Diseño y la Apariencia

CSS es un lenguaje de diseño gráfico que define **cómo** se van a mostrar los elementos HTML en la pantalla (colores, tipografías, espaciados, animaciones y distribución espacial).

El Modelo de Caja (*Box Model*)

Todo elemento en HTML es interpretado por el navegador como una caja rectangular compuesta por cuatro capas concéntricas:

1. **Content (Contenido):** El texto o imagen real dentro del elemento.

2. **Padding (Relleno):** El espacio interno entre el contenido y el borde.

3. **Border (Borde):** La línea que envuelve el relleno y el contenido.

4. **Margin (Margen):** El espacio externo que separa a la caja de otros elementos vecinos.

12) LA ARQUITECTURA WEB APLICADA EN EL SISTEMA DE GESTIÓN Y REGISTRO ESTUDIANTIL DE LA CASA DE LA CULTURA "CACIQUE GUANTAR"

El sistema de la Casa de la Cultura opera bajo la arquitectura Cliente-Servidor, la cual desacopla la interfaz de interacción del usuario del procesamiento intensivo de la lógica y la persistencia de datos:

- **El Cliente (Navegador Web / Lado del Cliente):** Es la interfaz receptora utilizada por los administrativos, profesores o representantes. Ejecuta el marcado semántico HTML, las reglas de estilo CSS y la interactividad ligera en el navegador del dispositivo (PC o teléfono móvil). Su función principal es capturar la información del estudiante (nombre, cédula, edad, disciplina artística) y enviarla mediante peticiones HTTP/HTTPS al servidor.

- **El Servidor (Back-end / Lado del Servidor):** Hospedado en un entorno de servidor web (Apache/Nginx local o remoto). Es la central de cómputo donde se ejecuta el motor de PHP y el framework Laravel, encargado de auditar la seguridad, aplicar las reglas del negocio y gestionar las consultas hacia la base de datos.

Descomposición de la Arquitectura por Capas

La aplicación aplica la arquitectura N-Capas para separar responsabilidades y garantizar la mantenibilidad del código a largo plazo:

Capa 1: Presentación (Front-end)

Construida con HTML5 y CSS3 responsivo (utilizando clases utilitarias de maquetación). Presenta los formularios de inscripción unificada y los módulos de consulta. Garantiza la accesibilidad y el diseño adaptativo (*Mobile First*) para que el personal pueda registrar o verificar alumnos desde cualquier pantalla.

Capa 2: Lógica de Negocio (Back-end)

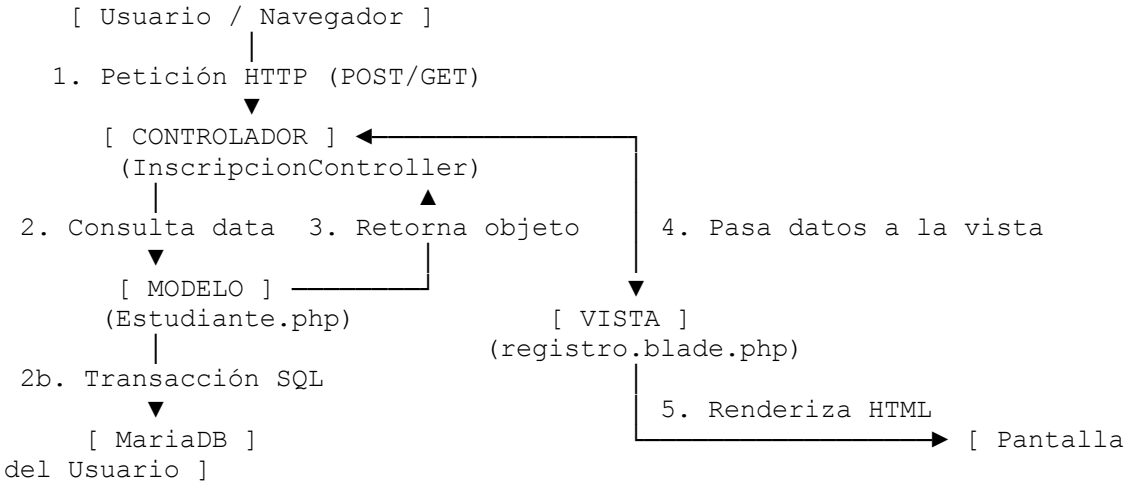
Desarrollada en PHP 8.x bajo el framework Laravel. En esta capa residen las validaciones de datos (verificar que la cédula del alumno no esté duplicada, comprobar el límite de cupos en talleres como música, danza o ajedrez) y el control de sesiones de usuario.

Capa 3: Persistencia de Datos (Database Layer)

Gestionada mediante el motor relacional MariaDB. La capa de datos organiza los registros en tablas normalizadas (Estudiantes, Representantes, Disciplinas, Inscripciones) unidas mediante claves primarias y foráneas, garantizando la integridad referencial y eliminando la duplicidad de información.

El Patrón Modelo-Vista-Controlador (MVC) en Laravel

El desarrollo del sistema implementa de forma estricta el patrón MVC, organizando la estructura interna del código en tres componentes principales:



La Vista (View)

Son los archivos de plantilla motorizados por Blade (`registro.blade.php`). Contienen la estructura visual del formulario de inscripción. No realizan consultas directas a la base de datos ni contienen lógica compleja; solo reciben variables y renderizan los elementos dinámicos que el usuario ve en pantalla.

B. El Controlador (Controller)

Es la clase intermedia (ejemplo: `InscripcionController.php`). Cuando un usuario hace clic en el botón "Finalizar Registro":

1. El Controlador intercepta los datos enviados por la petición HTTP `$request`.

2. Ejecuta la validación de campos obligatorios (cédula, disciplina, representante).

3. Invoca al Modelo para procesar el almacenamiento.

4. Redirecciona al usuario a una vista de éxito o retorna un reporte en pantalla.

El Modelo (Model)

Son las clases de Laravel que representan las entidades físicas de la institución (ejemplo: `Estudiante.php` o `Inscripcion.php`). Utilizan el ORM Eloquent para abstraer las consultas SQL. Permiten interactuar con las tablas de MariaDB mediante código orientado a objetos fluido (ejemplo: `Estudiante::create($datosValidados);`) sin necesidad de escribir SQL puro de forma repetitiva.

Justificación Técnica e Impacto Institucional

La implementación de esta arquitectura web aporta soluciones técnicas concretas al diagnóstico realizado en la institución:

1. **Seguridad de la Data:** La centralización en la capa del servidor mediante Laravel protege al sistema contra vulnerabilidades comunes de la web, como inyecciones SQL (gracias a sentencias preparadas de PDO) y ataques CSRF (Falsificación de Solicitud en Sitios Cruzados).

2. **Eficiencia Operativa:** El procesamiento desacoplado en tres capas sustituye los archivos de papel por transacciones algorítmicas, reduciendo el tiempo de inscripción y búsqueda de expedientes de horas a segundos.

3. **Mantenibilidad y Escalabilidad:** Si en el futuro la Casa de la Cultura decide incorporar un nuevo módulo académico (ej. artes plásticas o teatro) o integrar una API para pagos/constancias digitales, la arquitectura MVC permite extender el código existente sin alterar las bases del sistema.